EEE 313 INTRODUCTION TO EMBEDDED SYSTEMS
FINAL EXAM PROJECT HOMEWORKS

(25P) Q1. Perform an application which must include the following elements with your board (Raspberry Pi Pico, Raspberry Pi Model 3/4/5, or STM32): At least (i) one RGB LED, (ii) one potentiometer (except the one for LCD), (iii) two buttons, (iv) one LCD, (v) one buzzer, (vi) one Sensor, and (vii) one Motor. Note that you must use your assigned board, motor, sensor, RGB LED. Also, you can use more than the required number and mentioned elements above, i.e, LEDs, any resistor, any driver etc.

Purpose of Application: You are free about the purpose of application. Do not forget to mention your intended purpose (i.e, what you are trying to do for each situation) in your video capture and in this document.

Circuit Diagram: You are free to build your circuit for application. Draw your circuit in **Fritzing.**

Restriction: Neither the Arduino IDE nor the Arduino programming language will be used when performing this application. Use only Thonny IDE with MicroPhython or STM32CubeIDE with C/C++.

Homework Submission: Record a video with all the team members for your application. In your video content; explain your program codes, show your program to be compiled successfully, show your program to be uploaded to your board, show your circuit to be run successfully for each case. Note that no simulation study is requested.

The following files need to be uploaded to Teams.

1. This word document by completing the ANSWERS section (do not upload as pdf)
2. Your video file (Will be talked in English). Do NOT speak in Turkish! Show your face!)
3. Fritzing circuit file
4. Application project folder created by IDE software. Include your source file

Deadline for submission: 4th January, 2026. Note that 5 points from 25 will be deducted for each day of late uploading!. The system will be closed 4 days later than the deadline!

----------ANSWERS----------

Project Team: Salih Gündüz, Mustafa Burak Başar

Your Board: STMicroelectronics STM32F407G-DISC1

Type of Your Motor: Brushed DC

Your Sensor Type: Soil Moisture Sensor

Type of your RGB LED: Common Cathode.

Your Software IDE: STM32CubeIDE

Your Programming Language: C/C++.

Application Purpose:

The purpose of this project is to develop an intelligent and user-friendly soil moisture monitoring and control system for greenhouse farming. The system is designed to help farmers monitor the soil moisture of their crops automatically and apply irrigation only when it is needed. A soil moisture sensor continuously measures the moisture level of the soil, and the system processes this data to decide whether the plants require water.

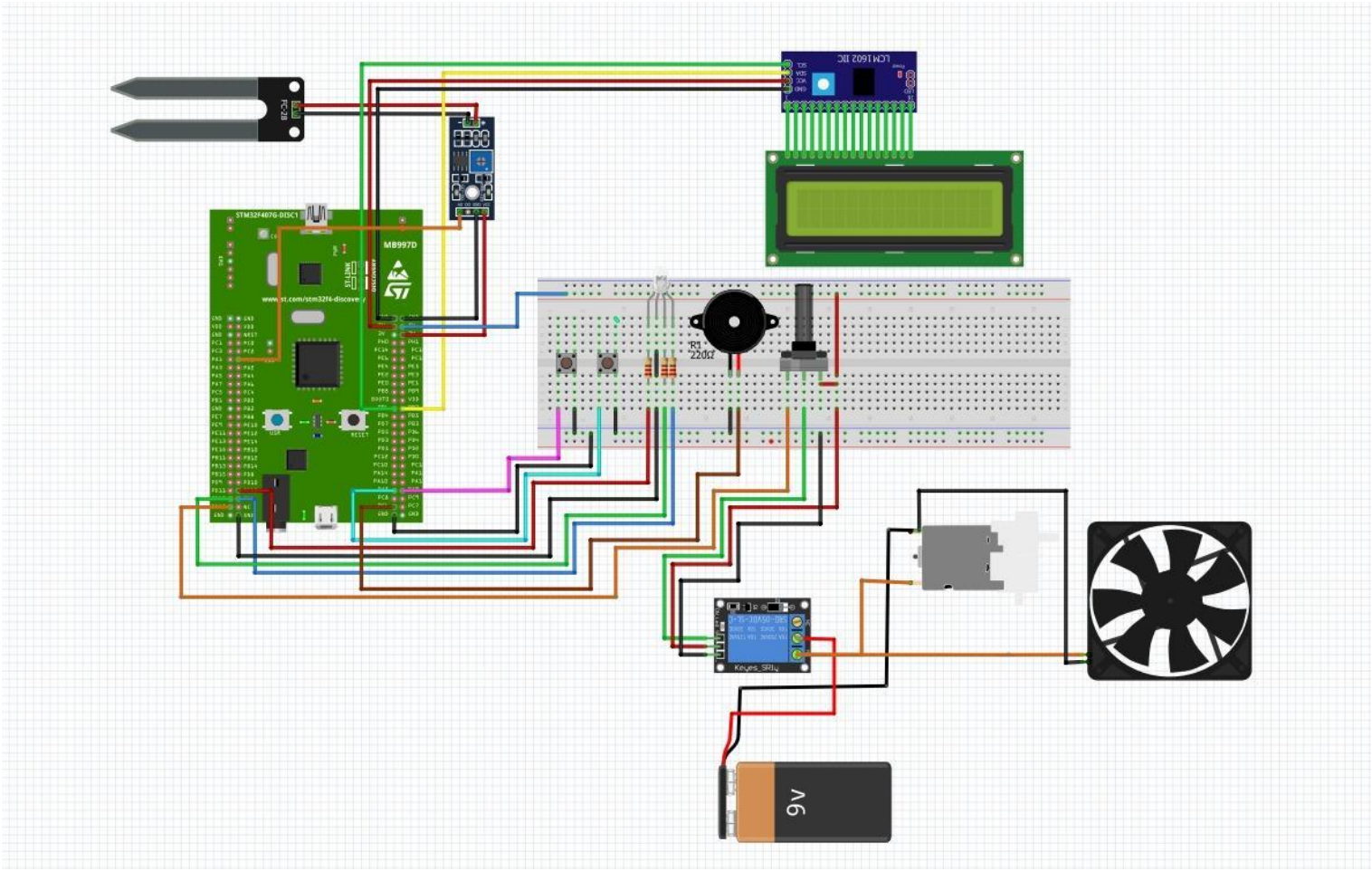
When the soil is dry, the system activates the water pump through a relay module and starts irrigation automatically. When the soil moisture reaches a normal or wet level, the pump is turned off to prevent overwatering. In addition to automatic operation, the system also includes a manual control mode, allowing the user to control the motor directly using buttons. Furthermore, a potentiometer-based emergency control mechanism is included, which enables the user to quickly stop the motor in emergency situations by adjusting the voltage level.

The system provides clear visual warnings using an RGB LED. Different colors indicate different soil conditions, allowing the user to understand the system status at a glance. An audible warning system is implemented using a buzzer, which alerts the user when the soil is too dry or too wet. These warning mechanisms increase system safety and user awareness.

To improve usability, a 16x2 LCD display is used as a simple and effective user interface. The LCD shows real-time soil moisture values, moisture percentage, system mode (automatic or manual), motor status, and soil condition. This display acts as a small control panel, making the system easy to understand and operate even for non-technical users.

Overall, this project aims to increase water efficiency, support healthy plant growth, reduce manual labor, and provide a low-cost, practical, and reliable smart agriculture **solution** for greenhouse applications.

Fritzing Circuit Diagram:



Program codes:

```c
#include "main.h"
#include "lcd_i2c.h"
#include <stdio.h>

/* ===================== TYPEDEFS ===================== */
typedef enum {
  MODE_AUTO = 0,
  MODE_MANUAL = 1
} ControlMode;

typedef enum {
  SOIL_DRY = 0,
  SOIL_NORMAL = 1,
  SOIL_WET = 2
} SoilStatus;

/* ===================== DEFINES ===================== */
#define RELAY_ACTIVE_LOW   1

#if RELAY_ACTIVE_LOW
  #define RELAY_ON()   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET)
  #define RELAY_OFF()  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET)
#else
  #define RELAY_ON()   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET)
  #define RELAY_OFF()  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET)
#endif

// RGB
#define R_ON()   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET)
#define R_OFF()  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_RESET)
#define G_ON()   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET)
#define G_OFF()  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET)
#define B_ON()   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET)
#define B_OFF()  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET)

// Buzzer
#define BUZZER_ON()   HAL_GPIO_WritePin(GPIOC, GPIO_PIN_6, GPIO_PIN_SET)
#define BUZZER_OFF()  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_6, GPIO_PIN_RESET)

// Buttons
#define BTN1_PRESSED() (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET)
#define BTN2_PRESSED() (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_RESET)

// Timing
#define LCD_UPDATE_MS      250
#define CONTROL_UPDATE_MS  200
#define BUZZER_BEEP_MS     250
#define BTN_DEBOUNCE_MS     60

// ---- SOIL ADC THRESHOLDS (OPTIMIZE EDILDI) ----
#define SOIL_ADC_DRY_TH     3200   // kuru
#define SOIL_ADC_NORMAL_TH  2500   // normal
#define SOIL_ADC_WET_TH     2000   // islak
```

```c
// ---- ADC FILTER ----
#define SOIL_FILTER_SIZE   4

/* ===================== HANDLES ===================== */
ADC_HandleTypeDef hadc1;
I2C_HandleTypeDef hi2c1;

/* ===================== GLOBALS ===================== */
static ControlMode g_mode = MODE_AUTO;
static SoilStatus  g_status = SOIL_NORMAL;

static uint8_t g_pump_manual = 0;
static uint8_t g_pump_state  = 0;
static uint8_t g_buzzer_state = 0;

static uint32_t g_lastLcdMs = 0;
static uint32_t g_lastCtlMs = 0;
static uint32_t g_lastBeepMs = 0;

static uint32_t g_btn1LastMs = 0;
static uint32_t g_btn2LastMs = 0;
static uint8_t  g_btn1Prev = 0;
static uint8_t  g_btn2Prev = 0;

static uint16_t adc_soil = 0;
static uint8_t  soil_percent = 0;

// Filter variables
static uint16_t soil_buf[SOIL_FILTER_SIZE];
static uint8_t  soil_idx = 0;
static uint32_t soil_sum = 0;

/* ===================== PROTOTYPES ===================== */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_I2C1_Init(void);

static void rgb_set(SoilStatus st);
static uint16_t read_adc_channel(uint32_t channel);
static uint16_t soil_filter(uint16_t sample);
static uint8_t soil_adc_to_percent(uint16_t adc);
static void control_update(void);
static void lcd_update(void);
static void buttons_update(void);

/* ===================== FUNCTIONS ===================== */

static void rgb_set(SoilStatus st)
{
  R_OFF(); G_OFF(); B_OFF();

  if (st == SOIL_DRY) {
    R_ON();        // 🔴 KURU
  }
  else if (st == SOIL_NORMAL) {
    G_ON();        // 🟢 NORMAL
  }
  else {
    B_ON();        // 🔵 ISLAK
  }
}
```

```c
static uint16_t read_adc_channel(uint32_t channel)
{
  ADC_ChannelConfTypeDef sConfig = {0};

  sConfig.Channel = channel;
  sConfig.Rank = 1;
  sConfig.SamplingTime = ADC_SAMPLETIME_480CYCLES;
  HAL_ADC_ConfigChannel(&hadc1, &sConfig);

  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, 50);
  uint16_t val = HAL_ADC_GetValue(&hadc1);
  HAL_ADC_Stop(&hadc1);

  return val;
}

static uint16_t soil_filter(uint16_t sample)
{
  soil_sum -= soil_buf[soil_idx];
  soil_buf[soil_idx] = sample;
  soil_sum += sample;

  soil_idx++;
  if (soil_idx >= SOIL_FILTER_SIZE)
    soil_idx = 0;
  return (uint16_t)(soil_sum / SOIL_FILTER_SIZE);
}

static uint8_t soil_adc_to_percent(uint16_t adc)
{
  const uint16_t ADC_DRY = 3200;   // kuru kalibrasyon
  const uint16_t ADC_WET = 2000;   // ıslak kalibrasyon

  if (adc >= ADC_DRY) return 0;
  if (adc <= ADC_WET) return 100;

  return (uint8_t)(((ADC_DRY - adc) * 100) / (ADC_DRY - ADC_WET));
}

static void control_update(void)
{
  uint16_t raw = read_adc_channel(ADC_CHANNEL_0);
  adc_soil = soil_filter(raw);
  soil_percent = soil_adc_to_percent(adc_soil);

  if (adc_soil >= SOIL_ADC_DRY_TH)
    g_status = SOIL_DRY;
  else if (adc_soil >= SOIL_ADC_NORMAL_TH)
    g_status = SOIL_NORMAL;
  else
    g_status = SOIL_WET;

  if (g_mode == MODE_AUTO)
    g_pump_state = (g_status == SOIL_DRY);
  else
    g_pump_state = g_pump_manual;

  g_pump_state ? RELAY_ON() : RELAY_OFF();
  rgb_set(g_status);

  if (g_status == SOIL_DRY || g_status == SOIL_WET) {
    uint32_t now = HAL_GetTick();
    if (now - g_lastBeepMs >= BUZZER_BEEP_MS) {
      g_lastBeepMs = now;
      g_buzzer_state ^= 1;
      g_buzzer_state ? BUZZER_ON() : BUZZER_OFF();
    }
```

```
    }
  } else {
    g_buzzer_state = 0;
    BUZZER_OFF();
  }
}

static void lcd_update(void)
{
  char l1[17], l2[17];

  snprintf(l1, sizeof(l1), "Soil:%4u %3u%%", adc_soil, soil_percent);

  const char *m = (g_mode == MODE_AUTO) ? "AUTO" : "MAN";
  const char *p = g_pump_state ? "ON " : "OFF";
  const char *s = (g_status == SOIL_DRY) ? "DRY" :
                  (g_status == SOIL_NORMAL) ? "NOR" : "WET";

  snprintf(l2, sizeof(l2), "%s P:%s %s", m, p, s);

  lcd_put_cur(0,0); lcd_send_string("                ");
  lcd_put_cur(0,0); lcd_send_string(l1);
  lcd_put_cur(1,0); lcd_send_string("                ");
  lcd_put_cur(1,0); lcd_send_string(l2);
}

static void buttons_update(void)
{
  uint32_t now = HAL_GetTick();

  uint8_t b1 = BTN1_PRESSED();
  if (b1 && !g_btn1Prev && (now - g_btn1LastMs > BTN_DEBOUNCE_MS)) {
    g_btn1LastMs = now;
    if (g_mode == MODE_MANUAL) g_pump_manual ^= 1;
  }
  g_btn1Prev = b1;

  uint8_t b2 = BTN2_PRESSED();
  if (b2 && !g_btn2Prev && (now - g_btn2LastMs > BTN_DEBOUNCE_MS)) {
    g_btn2LastMs = now;
    g_mode ^= 1;
    if (g_mode == MODE_AUTO) g_pump_manual = 0;
  }
  g_btn2Prev = b2;
}

/* ==================== MAIN ==================== */
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_ADC1_Init();
  MX_I2C1_Init();

  RELAY_OFF(); BUZZER_OFF();
  R_OFF(); G_OFF(); B_OFF();

  lcd_init();
  lcd_put_cur(0,0); lcd_send_string("Smart Irrigation");
  lcd_put_cur(1,0); lcd_send_string("Starting...");
  HAL_Delay(800);

  g_lastLcdMs = HAL_GetTick();
  g_lastCtlMs = HAL_GetTick();

  while (1)
  {
    buttons_update();
    uint32_t now = HAL_GetTick();

    if (now - g_lastCtlMs >= CONTROL_UPDATE_MS) {
      g_lastCtlMs = now;
      control_update();
    }
```

```
271
272     if (now - g_lastLcdMs >= LCD_UPDATE_MS) {
273       g_lastLcdMs = now;
274       lcd_update();
275     }
276
277     HAL_Delay(5);
278   }
279 }
280
281
282 /* ---------------------- Peripheral Init ---------------------- */
283
284 void SystemClock_Config(void)
285 {
286   RCC_OscInitTypeDef RCC_OscInitStruct = {0};
287   RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
288
289   __HAL_RCC_PWR_CLK_ENABLE();
290   __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
291
292   RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
293   RCC_OscInitStruct.HSEState = RCC_HSE_ON;
294   RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
295   RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
296   RCC_OscInitStruct.PLL.PLLM = 8;
297   RCC_OscInitStruct.PLL.PLLN = 336;
298   RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2; // 168 MHz
299   RCC_OscInitStruct.PLL.PLLQ = 7;
300
301   if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
302     Error_Handler();
303   }
304
305   RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK |
306                                 RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
307   RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
308   RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
309   RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4; // 42 MHz
310   RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2; // 84 MHz
311
312   if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK) {
313     Error_Handler();
314   }
315 }
316
317 static void MX_ADC1_Init(void)
318 {
319   ADC_ChannelConfTypeDef sConfig = {0};
320
321   __HAL_RCC_ADC1_CLK_ENABLE();
322
323   hadc1.Instance = ADC1;
324   hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
325   hadc1.Init.Resolution = ADC_RESOLUTION_12B;
326   hadc1.Init.ScanConvMode = ENABLE;
327   hadc1.Init.ContinuousConvMode = DISABLE;
328   hadc1.Init.DiscontinuousConvMode = DISABLE;
329   hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
330   hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
331   hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
332   hadc1.Init.NbrOfConversion = 2;
333   hadc1.Init.DMAContinuousRequests = DISABLE;
334   hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
335
336   if (HAL_ADC_Init(&hadc1) != HAL_OK) {
337     Error_Handler();
338   }
```

```c
  // Rank 1: Channel 0 (PA0)
  sConfig.Channel = ADC_CHANNEL_0;
  sConfig.Rank = 1;
  sConfig.SamplingTime = ADC_SAMPLETIME_144CYCLES;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK) {
    Error_Handler();
  }

  // Rank 2: Channel 1 (PA1)
  sConfig.Channel = ADC_CHANNEL_1;
  sConfig.Rank = 2;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK) {
    Error_Handler();
  }
}

static void MX_I2C1_Init(void)
{
  __HAL_RCC_I2C1_CLK_ENABLE();

  hi2c1.Instance = I2C1;
  hi2c1.Init.ClockSpeed = 100000;
  hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
  hi2c1.Init.OwnAddress1 = 0;
  hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2 = 0;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;

  if (HAL_I2C_Init(&hi2c1) != HAL_OK) {
    Error_Handler();
  }
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();

  /* ================= RGB LEDLER (PD12, PD13, PD14) ================= */
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14, GPIO_PIN_RESET);

  GPIO_InitStruct.Pin = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;   // PUSH-PULL
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

  /* ================= RÖLE (PD15) ================= */
  // OPEN-DRAIN + harici 10k pull-up (IN -> 5V)
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET); // OFF / Hi-Z

  GPIO_InitStruct.Pin = GPIO_PIN_15;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;   // 🟢 OPEN-DRAIN
  GPIO_InitStruct.Pull = GPIO_NOPULL;           // harici pull-up var
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

  /* ================= BUZZER (PC6) ================= */
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_6, GPIO_PIN_RESET);

  GPIO_InitStruct.Pin = GPIO_PIN_6;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /* ================= BUTTONLAR (PA8, PA9) ================= */
  GPIO_InitStruct.Pin = GPIO_PIN_8 | GPIO_PIN_9;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

  // I2C pins (PB6/PB7) genelde IOC üzerinden AF4 ayarlanır.
  // Burada elle init etmiyoruz; CubeMX otomatik yapar.
```

```
422
423 /* USER CODE BEGIN 4 */
424 void Error_Handler(void)
425 {
426   __disable_irq();
427   while (1)
428   {
429     // Error blink: Red LED
430     R_ON();
431     HAL_Delay(150);
432     R_OFF();
433     HAL_Delay(150);
434   }
435 }
436 /* USER CODE END 4 */
437
438 #ifdef  USE_FULL_ASSERT
439 void assert_failed(uint8_t *file, uint32_t line)
440 {
441   (void)file;
442   (void)line;
443 }
444 #endif
```

Photo for your circuit (only 1 photo):